

WEEK-12

TASK-1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and System throughput under different buffer sizes

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <semaphore.h>

#include <time.h>

#define ITEMS 100000

int *buffer;

int buffer_size;

int in = 0;

int out = 0;

sem_t empty;

sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)

{

    for (int i = 0; i < ITEMS; i++)

    {

        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = i;

        in = (in + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);

    }

    return NULL;

}

void *consumer(void *arg)
```

```

{
    long long consumed_sum = 0;
    for (int i = 0; i < ITEMS; i++)
    {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        int item = buffer[out];

        out = (out + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);

        consumed_sum += item;
    }

    printf("Consumer sum = %lld\n", consumed_sum);

    return NULL;
}

void run_test(int size)
{
    pthread_t producer_thread;

    pthread_t consumer_thread;

    buffer_size = size;

    buffer = malloc(buffer_size * sizeof(int));

    if (buffer == NULL)
    {
        perror("malloc");

        exit(1);
    }

    in = 0;

    out = 0;

    sem_init(&empty, 0, buffer_size);

    sem_init(&full, 0, 0);

    pthread_mutex_init(&mutex, NULL);

    clock_t start = clock();

    pthread_create(&producer_thread, NULL, producer, NULL);

```

```

pthread_create(&consumer_thread, NULL, consumer, NULL);

pthread_join(producer_thread, NULL);

pthread_join(consumer_thread, NULL);

clock_t end = clock();

double time_taken =

    (double)(end - start) / CLOCKS_PER_SEC;

double throughput = ITEMS / time_taken;

printf("Buffer Size : %d\n", buffer_size);

printf("Items      : %d\n", ITEMS);

printf("Time       : %.6f seconds\n", time_taken);

printf("Throughput : %.2f items/second\n", throughput);

printf("-----\n");

sem_destroy(&empty);

sem_destroy(&full);

pthread_mutex_destroy(&mutex);


free(buffer);
}

int main()
{
    printf("Producer-Consumer Problem\n");
    printf("=====\n\n");

    run_test(1);

    run_test(5);

    run_test(10);

    run_test(50);

    run_test(100);

    return 0;
}

```

```

root@LAPTOP-RJFD8EBT:~/inode_test# nano producer_consumer.c
root@LAPTOP-RJFD8EBT:~/inode_test# gcc producer_consumer.c -o producer_consumer -pthread
root@LAPTOP-RJFD8EBT:~/inode_test# ./producer_consumer
Producer-Consumer Problem
=====
Consumer sum = 4999950000
Buffer Size : 1
Items       : 100000
Time        : 6.538807 seconds
Throughput  : 15295.18 items/second
=====
Consumer sum = 4999950000
Buffer Size : 5
Items       : 100000
Time        : 1.150192 seconds
Throughput  : 86942.01 items/second
=====
Consumer sum = 4999950000
Buffer Size : 10
Items       : 100000
Time        : 0.641038 seconds
Throughput  : 155992.13 items/second
=====
Consumer sum = 4999950000
Buffer Size : 50
Items       : 100000
Time        : 0.062806 seconds
Throughput  : 1592204.57 items/second
=====
Consumer sum = 4999950000
Buffer Size : 100
Items       : 100000
Time        : 0.043251 seconds
Throughput  : 2312085.27 items/second
=====
root@LAPTOP-RJFD8EBT:~/inode_test#

```

TASK-2:

Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```

root@LAPTOP-RJFD8EBT:~# nano deadlock.c
root@LAPTOP-RJFD8EBT:~# gcc deadlock.c -o deadlock -pthread
root@LAPTOP-RJFD8EBT:~# ./deadlock
Deadlock Demonstration
=====
Thread 1: Trying to lock Resource 1...
Thread 1: Locked Resource 1.
Thread 2: Trying to lock Resource 2...
Thread 2: Locked Resource 2.
Thread 1: Trying to lock Resource 2...
Thread 2: Trying to lock Resource 1...
^C
root@LAPTOP-RJFD8EBT:~# nano deadlock_prevention.c
root@LAPTOP-RJFD8EBT:~# gcc deadlock_prevention.c -o deadlock_prevention -pthread
root@LAPTOP-RJFD8EBT:~# ./deadlock_prevention
Deadlock Prevention Using Resource Ordering
=====
Thread 1: Locking Resource 1...
Thread 1: Resource 1 locked.
Thread 2: Locking Resource 1...
Thread 1: Locking Resource 2...
Thread 1: Resource 2 locked.
Thread 1: Using both resources.
Thread 1: Resources released.
Thread 2: Resource 1 locked.
Thread 2: Locking Resource 2...
Thread 2: Resource 2 locked.
Thread 2: Using both resources.
Thread 2: Resources released.
Program completed without deadlock.
root@LAPTOP-RJFD8EBT:~# nano deadlock.c
root@LAPTOP-RJFD8EBT:~# gcc deadlock.c -o deadlock -pthread
./deadlock
Deadlock Demonstration
=====
Thread 1: Trying to lock Resource 1...
Thread 1: Locked Resource 1.
Thread 2: Trying to lock Resource 2...
Thread 2: Locked Resource 2.
Thread 1: Trying to lock Resource 1...
Thread 1: Trying to lock Resource 2...
^C
root@LAPTOP-RJFD8EBT:~# |

```